

AN2DL - First Challenge Report

Bobert

Filippo Maria Alemagna, Alessandro Mancuso, Luca Trapella, Xuwen Ye

flippomariaalemagna, alemancusopoli, lucatrapella, xuwen1ye

277903, 277818, 279894, 277338

November, 2025

1 Introduction

In this project we tackle the **Pirate Pain Dataset** in a supervised learning problem based on *multivariate time series classification*. Each sample consists of short temporal sequences and static attributes of synthetic pirate characters, with the goal of predicting one of three pain levels.

The main objective is to design and evaluate a neural architecture capable of capturing both temporal dynamics and static contextual features. To obtain the best possible performance, we followed a structured strategy: (i) conducting data analysis to understand the dataset and derive implications for model design, (ii) pre-processing data to prepare consistent inputs for training, (iii) building the most suitable architecture for the task, (iv) performing an initial training phase to detect underfitting or overfitting and refining the model accordingly, and (v) tuning the network parameters to improve generalization.

2 Problem Analysis [4]

The training set shows a clear **class imbalance**, with the majority of samples labeled as *no_pain*. This motivated the use of metrics such as the **F1-score**, which are more informative than accuracy under uneven class distributions.

A second issue concerns the strong **dynamic vari-**

ability between the columns of each sample: different joints operate on different scales and exhibit heterogeneous variance. This made *z-score normalization* more suitable than min-max scaling. During this analysis, we identified two **non-informative features**: *joint_30*, which was constant across all time steps and all pirates, and *time*, which followed an identical linear trend for every sequence.

The temporal signals themselves showed **weak autocorrelation** and minimal cross-correlation (even though some pirates showed high cross-correlation between some pairs of joints, the mean cross-validation matrix showed no significant patterns). Many sequences resembled noisy processes, with a sharp autocorrelation drop after lag = 0 and no clear temporal dependency. This implied a high risk of overfitting when using recurrent architectures and suggested the potential need for **feature extraction** to reduce noise and improve signal relevance.

Based on these observations, our initial assumptions for model design and training were: (i) using a **recurrent module** to capture whatever temporal structure is present; (ii) applying **K-fold cross-validation** with stratified splits to face class imbalance and reduce variance in model evaluation; and (iii) treating the dataset as a combination of static and dynamic attributes.

In particular, the static features (*n_hands*, *n_eyes*, *n_legs*) needed to be processed separately from the multivariate time series. This motivated the con-

struction of a **hybrid architecture** composed of a recurrent branch for temporal data and an MLP branch for static attributes, combined before the final classification layer.

3 Method

3.1 Data Preprocessing

The preprocessing pipeline was designed to convert the raw tabular data into a consistent format suitable for a hybrid neural architecture. Both training and test CSV files were first loaded and converted into numerical dataframes with consistent column ordering. The categorical static attributes (*n_legs*, *n_hands*, *n_eyes*) were transformed into one-hot vectors so they could be processed by the MLP branch. Non-informative columns were removed: *joint_30*, which was constant across samples, and *time*, which showed the same uniform progression for all pirates. Only dynamic features were then normalized, applying z-score standardization within each fold using the statistics of the corresponding training split, while the test set was normalized using the same values to avoid leakage.

After normalization, the multivariate time series were reshaped into fixed-length windows through a dedicated sequence-construction function. Following the recommendations reported in the Logbook (11/11), the window size was selected based on the very short autocorrelation observed in the joint trajectories.

Finally, custom dataloaders were created to pair each time-series window with its associated static one-hot features and to generate batched inputs compatible with the hybrid architecture.

3.2 Model building and training

Our first modeling strategy relied on a **hybrid architecture** as discussed before. Both branches were designed to produce output vectors of the same dimensionality so that their embeddings could be concatenated without adding unnecessary projection layers.

For the recurrent branch, we implemented several variants, including GRU- and LSTM-based networks. To control model capacity and reduce overfitting, we employed small hidden sizes, dropout, batch normalization, and careful weight initializa-

tion. The overall training loop was based on **stratified K-fold cross-validation** with shuffling: at every fold, the model was re-initialized and trained from scratch with early stopping, ensuring balanced class proportions and reliable validation estimates. A second architectural direction was introduced by replacing the recurrent backbone with a **1D convolutional module**. This choice was motivated by the dataset’s limited temporal dependencies and weak autocorrelation: the short windows and high noise levels suggested that recurrent units might be unnecessarily complex. The CNN-based architecture therefore used a shallow **1D convolution** with small kernels to extract local temporal patterns, followed by an **attention layer** to help the model focus on the most relevant segments of the series. All remaining components of the pipeline were kept identical to ensure a fair comparison between recurrent and convolutional architectures.

3.3 Predicting on the Test Set

The final predictions on the Test Set were computed using majority voting in two steps:

- we compute the majority vote for each window between the different k folds,
- we compute the majority vote for each pirate by looking at the windows representing the same time series, obtaining a final vote for each one.

4 Experiments

All models were trained under a unified configuration to ensure comparability [2] [3] [5] [6].

Table 1: Model comparison using F1-score in validation. Best results in **bold**.

Model	F1-score
LSTM + MLP	0.9278 $\pm$ 0.0062
GRU + MLP	0.9301 $\pm$ 0.0124
BiGRU + MLP	0.9261 $\pm$ 0.0091
1DConv + Att + MLP	0.9343 $\pm$ 0.0097

Recurrent models achieved low **low F1-scores** due to strong **overfitting**: their high parameter

count is unnecessary for short, noisy sequences, causing them to memorize fluctuations rather than learn meaningful temporal patterns. The bidirectional variant did not help, as the data shows minimal temporal structure in either direction, only increasing model complexity and overfitting.

The **1D CNN + Attention** architecture performed slightly better thanks to its simpler local filters, but overfitting was still present, indicating that the raw joint trajectories contain too much noise. This motivated a shift toward using **statistical features** and treating the input more like static data.

To obtain more informative and stable representations, we replaced raw joint trajectories with their **temporal derivatives** and extracted **additional statistical descriptors**: counts of each *pain_survey* value, and the mean, standard deviation, and peak of the computed *joint_speed*. Joint time series with variance below a dataset-wide threshold were removed, as they carried no meaningful temporal information.

Finally, the MLP branch was expanded to better process the enlarged set of static features, and BatchNorm was replaced by **LayerNorm**, which is more suitable for feed-forward architectures [7].

5 Results [1]

The new model (**Features + 1DConv + Att + MLP**) achieved a weighted average F1 score on the validation set of **0.9473 $\pm$ 0.0130**.

This result points out an improvement over the previous models.

Anyway, we expected the modifications to be more impactful and hoped to reach an even better F1 score.

6 Discussion

The performance achieved by the final model shows that it reached a good generalization. This result comes from discarding noisy time series and replacing them with compact statistical representations that carry the relevant information. Moreover, we managed to build a fairly simple model, with a limited number of parameters.

The main limitations of our approach stem from the small size and high noise level of the dataset. Removing outliers using z-score and Isolation Forest [4] did not improve generalization: although validation performance remained stable (≈ 0.93), test performance dropped, suggesting that these points represent meaningful variability rather than noise. A further weakness is the absence of an extensive hyperparameter grid search, which may have restricted the model’s final performance. As a final consideration, during the training of our best model, a **slight underfitting** issue can be noticed. This problem however, is still better than having an overfitting model, and can be easily fixed by a future grid search.

7 Conclusions

In this project, we faced the Pirate Pain classification task by combining time-series and static features. Due to the noisy and weakly correlated nature of the joint trajectories, derivative-based representations and statistical descriptors proved more effective than raw sequences. The resulting hybrid model, built on a 1D CNN with attention and a static feature set, achieved a robust and satisfying performance.

The main limitations concern the small dataset, the high noise level, and the absence of an extensive hyperparameter search. Future improvements may include more advanced feature extraction and broader tuning to further enhance generalization.

References

- [1] Notebook best model.
- [2] Notebook bigru model.
- [3] Notebook cnn model.
- [4] Notebook data analysis.
- [5] Notebook gru model.
- [6] Notebook lstm model.
- [7] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization, 2016.